

От опкода к намерению: как превратить идею «программирование никогда уже не будет прежним» в запоминающийся двадцатиминутный доклад

Главный вывод исследования

У вашей идеи есть очень сильное историческое основание, но я бы **слегка поменял научную формулировку тезиса**.

Не «программирование всё время увеличивает энтропию», а:

Вся история программирования — это история роста уровня абстракции и количества решений, которые программист делегирует нижележащему интерпретатору.

Или, гораздо лучше для сцены:

Мы всё время увеличивали количество смысла, приходящегося на единицу человеческого текста.

Для доклада я бы ввёл собственный термин **«семантическая плотность»** и сразу честно сказал, что это не академическая единица измерения:

$$\text{Semantic Density} \approx \frac{\text{решения об implementation, которые мы делегировали машине}}{\text{объём явно написанного человеком намерения}}$$

Это не метрика, которую следует пытаться посчитать. Это **модель для размышления**.

Причём здесь есть великолепный интеллектуальный твист для первых двух минут выступления: **слово «энтропия» лучше использовать именно как ловушку**.

В классической теории информации Клод Шеннон специально отделил information от meaning: его математическая теория рассматривает неопределённость выбора сообщения, а не его семантический смысл; на первой странице знаменитой статьи 1948 года он прямо исключает semantic aspects из инженерной задачи коммуникации. ¹

То есть начало доклада может звучать примерно так:

«Когда я начал готовить этот доклад, я решил, что придумал отличную научную концепцию: программирование всё время увеличивает энтропию смысла. Потом я перечитал Шеннона. Оказалось, Шеннон был бы категорически против. Поэтому следующие двадцать минут я буду научно обосновывать термин, который я только что придумал».

Смех, а следом — серьёзный тезис.

Самая сильная формулировка, которую я вынес из исследования

История выглядит не просто как

machine code → assembler → C → SQL → prompt.

Гораздо интереснее она выглядит так:

каждое поколение программирования переносит часть программы из текста, который пишет человек, в машину, которая этот текст понимает.

Компилятор знает, как распределить регистры.

Runtime знает, как управлять памятью.

SQL optimizer знает, в каком порядке соединять таблицы.

Framework знает, как устроена форма.

Design system знает, как выглядит кнопка.

LLM знает тысячи ранее существовавших способов сделать «обычную форму клиента».

Agent дополнительно знает repository, tools, tests и может сам проверить результат. Современные agent harnesses именно так и строятся: агент собирает контекст, совершает действие, проверяет результат и повторяет цикл. ²

Отсюда, на мой взгляд, **главная фраза всего доклада:**

Мы не сжали программу до одной фразы. Мы перенесли почти всю программу в интерпретатор.

Это объясняет всю историю — от ассемблера до LLM — одной идеей.

И у этой истории есть ещё более сильный второй акт.

Фред Брукс в *No Silver Bullet* ещё в 1986 году разделил сложность разработки на **accidental complexity** — сложности представления идеи машине — и **essential complexity** — сложность самой задачи и создаваемой нами концептуальной системы. Он утверждал, что огромная часть исторического выигрыша уже пришла от устранения искусственных трудностей: ограничений железа, неудобных языков и машинного времени. ³

А затем в разделе... **об искусственном интеллекте**, написанном сорок лет назад, Брукс сформулировал практически готовый финал вашего доклада:

“The hard thing about building software is deciding what one wants to say, not saying it.” 4

Это, пожалуй, **лучший найденный мной материал для вау-эффекта**.

Потому что LLM практически доводит эксперимент Брукса до предела:

А что произойдёт, если стоимость *saying it* приблизится к нулю?

Ответ:

Мы наконец увидим, насколько дорого стоит deciding what we want to say.

И вот здесь доклад перестаёт быть рассказом про Copilot и превращается в доклад **про фундаментальную смену bottleneck в программировании**.

Историческая линия, которая действительно работает

Я бы не подавал историю как «языки становились всё выше и выше». Это немного школьная версия истории computer science.

Гораздо сильнее:

На каждом этапе человек переставал принимать некоторый класс решений.

До языка: программой была конфигурация машины

Небольшая, но полезная историческая поправка: начинать рассказ словами «сначала программы писали машинными кодами на перфокартах» не совсем точно.

Например, ранний ENIAC первоначально программировался физической конфигурацией кабелей, переключателей и function tables; Computer History Museum описывает его программирование именно как настройку аппаратуры, а не ввод привычной stored program. 5

Это на самом деле **даже лучше для вашей истории**.

Первая картинка:

женщина-программист стоит внутри программы.

Человек буквально соединяет вычисление проводами.

Возможная реплика:

«Первая IDE была довольно суровой.
Чтобы сделать refactoring, иногда надо было встать со стула».

И ещё:

«Dependency injection тогда понимали буквально».

Это хороший ранний смех, после которого можно показать перфокарту.

IBM пишет, что у классической 80-column card при программировании одна карта обычно представляла одну строку кода. ⁶

Отсюда ещё одна сценическая шутка:

«В те времена у `git diff` был вес».

И:

«Уронил колоду — получил merge conflict с гравитацией».

Последнее, разумеется, шутка, а не исторический факт.

Ассемблер: имена вместо чисел

Следующий переход качественный.

Не только

```
00110110 ...
```

а

```
ADD  
MOV  
JMP
```

Человек по-прежнему думает довольно близко к машине, но ему уже не требуется помнить числовое кодирование операций и абсолютные адреса.

Здесь впервые можно показать ваш главный графический приём:

What I say ↑
What the machine decides ↑ ↑

На каждом следующем слайде правая часть растёт.

FORTRAN: машина начинает решать «как»

Именно FORTRAN прекрасно поддерживает вашу аргументацию.

Команда Backus начинала проект с целью дать программисту возможность описывать numerical procedure в компактной математически похожей форме, а translator уже должен был получить эффективную программу для IBM 704. Первая система FORTRAN создавалась примерно два с половиной года и потребовала порядка 18 person-years — хороший показатель того, сколько интеллектуальной работы пришлось впервые перенести из программиста **в compiler**. 7

Это очень важное изменение перспективы:

Мы не сделали программирование короче бесплатно. Мы один раз написали огромную программу, чтобы миллионы следующих программ могли быть короче.

Это повторяется всю историю.

Compiler.
OS.
Garbage collector.
Standard library.
Framework.
Database optimizer.
Cloud platform.
Foundation model.

Человеческий текст уменьшается потому, что принимающая его машина становится всё умнее и тяжелее.

У FORTRAN есть ещё прекрасная визуальная деталь: ранняя форма исходного текста буквально несла следы носителя — punch card. IBM-карты имели 80 columns, и fixed-format FORTRAN распределял назначение по колонкам; физический носитель повлиял на syntax языка. 8

Это даёт очень красивую дугу:

Раньше **hardware** формировал наш язык.
Теперь natural language начинает формировать software.

Dijkstra: каждый слой создаёт новую машину

Здесь появляется академическая формулировка, которая, возможно, станет лучшим интеллектуальным каркасом всей презентации.

В описании layered systems Дейкстра рассматривал уровни программного обеспечения как последовательность абстрактных машин: software на одном уровне фактически превращает машину A_i в машину A_{i+1} . В его THE multiprogramming system после перехода на следующий уровень детали нижележащих processors и storage перестают быть частью модели, с которой работает следующий слой. 9

То есть можно сказать:

Абстракция — это способ построить компьютер, которого физически не существует.

На CPU нет `foreach`.

На CPU нет `Customer`.

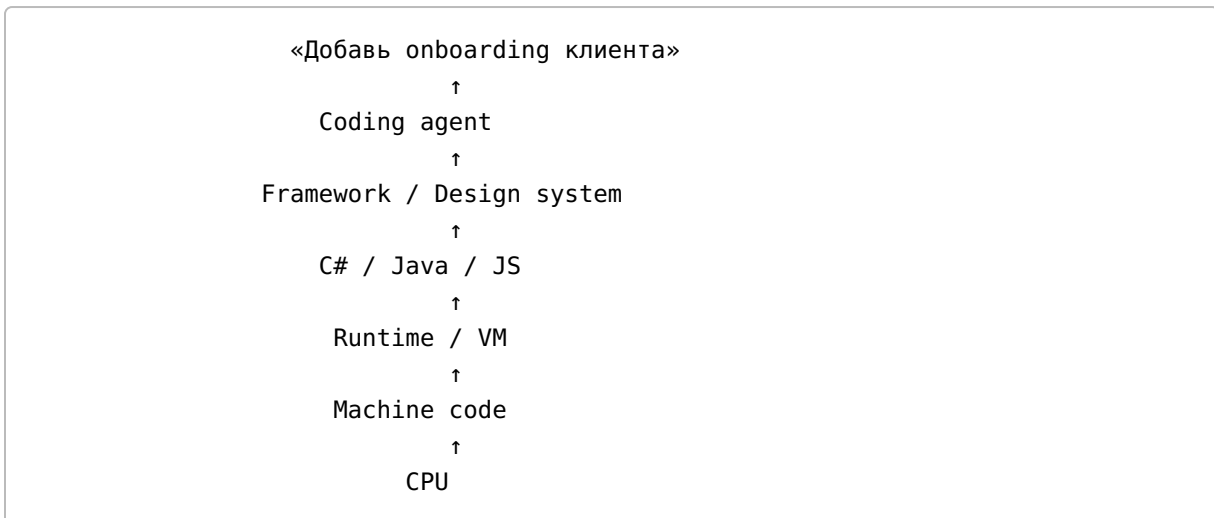
На CPU нет `HTTP request`.

На CPU нет `transaction`.

На CPU нет `button`.

Но каждый следующий слой создаёт **виртуальную машину понятий**, где эти сущности становятся примитивами.

И вот эту схему я бы буквально рисовал на слайде:



А затем сказать:

«Мы восемьдесят лет строили лестницу, чтобы разговаривать с компьютером всё меньше на его языке и всё больше на нашем».

Это, пожалуй, точнее вашего исходного тезиса, чем «эволюция языков».

Codd и SQL: алгоритм исчезает из запроса

Реляционная модель — отличный следующий скачок.

В статье Кодда 1970 года одним из центральных мотивов была independence программ от деталей машинного представления и организации данных. IBM, рассказывая историю relational databases, подчёркивает именно возможность получать данные без знания физического blueprint хранения.

10

Поэтому хороший пример:

```
SELECT customer_id
FROM orders
GROUP BY customer_id
HAVING COUNT(*) = 5;
```

Интерес здесь не в том, что SQL «похож на английский».

Это вообще не главный признак высокого уровня.

Главное:

программист больше не говорит, в каком порядке читать записи, какие pages трогать и какой physical access path использовать.

Из запроса исчезает большой кусок **algorithmic mechanism**.

Отсюда отличная фраза:

Высокоуровневый язык — это не язык с красивыми словами. Это язык, в котором вам разрешили перестать принимать некоторые решения.

Причём для технической аудитории я бы сделал маленькую корректировку исходного рассказа: BNF/EBNF определяют прежде всего **syntax/grammar**, а не полную semantics языка. В теории языков syntax и operational/denotational semantics рассматриваются отдельно. 11

Это лучше не превращать в отдельный слайд — просто не оставлять у аудитории повод спорить в комментариях.

Intentional Programming: мы уже пытались сохранить именно намерение

Очень сильная и относительно неизбитая историческая находка — Charles Simonyi и **Intentional Programming**.

Ещё в 1990-е Simonyi исследовал подход, при котором важным invariant должен оставаться **computational intent**, а конкретное представление и implementation могут эволюционировать независимо. На USENIX в 1997 году Intentional Programming описывался как «ecology for abstractions», позволяющая быстрее создавать domain-specific abstractions. ¹²

Это важно для вашего тезиса, потому что позволяет сказать:

«LLM не изобрели желание программировать намерениями.
Они впервые сделали достаточно универсальным интерпретатор этих намерений».

Вот это уже не стандартный рассказ «раньше assembly, теперь ChatGPT».

И даже «единица смысла» действительно существовала

Вы правильно вспомнили Артемия Лебедева.

В §148 «Ководства», опубликованном в 2008 году, Лебедев вводит **«единицу смысла»** как собственный практический способ думать о результативности дизайнера: ценность работы предлагается рассматривать не через потраченное время, а через появившиеся meaningful решения. Это, разумеется, не научная метрология, но для русскоязычной аудитории отсылка отличная. ¹³

Есть и более формальная историческая параллель: Function Point Analysis пытается измерять functional size software через предоставляемую пользователю business functionality, а не через количество строк исходного текста; IFPUG стандартизировал такой подход к functional sizing. ¹⁴

Можно пошутить:

«Оказывается, человечество много раз пыталось перестать измерять программистов килограммами исходников. Иногда даже успешно».

Почему coding agents — это не просто «ещё один язык более высокого уровня»

Вот здесь начинается настоящая новая часть вашего доклада.

Я бы специально сначала **защитил контраргумент**:

«Ну хорошо. Был assembler, появился FORTRAN. Был C, появился SQL. Теперь появился English. Что тут принципиально нового? Просто очередной уровень abstraction».

А потом показать: **нет, появились как минимум четыре качественных разрыва**.

Интерпретатор теперь понимает открытый язык

До LLM программист работал с относительно замкнутым языком.

У него есть syntax.

Есть определённые constructs.

Есть semantics.

Есть compiler/interpreter.

Не обязательно всё полностью deterministic на уровне всей программы, но допустимые выражения и их значение задаются формальной системой.

В prompt programming ситуация фундаментально другая. Исследователи, описывающие **promptware**, противопоставляют традиционным formal programming languages естественный язык как ambiguous, unstructured и context-dependent interface, выполняющийся на probabilistic runtime — LLM. ¹⁵

То есть:

```
customer.Name
```

имеет очень небольшой набор допустимых интерпретаций.

А:

```
Сделай красивую форму клиента
```

имеет практически неограниченное количество reasonable interpretations.

Получается парадокс:

Семантическая ёмкость растёт одновременно с семантической неопределённостью.

Это одна из основных идей выступления.

Prompt не содержит всей программы

Это, на мой взгляд, **самая интересная часть вашего тезиса.**

Возьмём:

«Добавь форму редактирования клиента в стиле остальных страниц приложения».

В этой фразе нет:

- CSS;
- структуры компонентов;
- routing;
- validation library;
- цвета;
- spacing;
- правил accessibility;
- error handling;
- naming conventions;
- state-management choice.

Но агент способен сделать разумный результат, потому что фактически его вход не равен только этой фразе.

Для слайда я бы написал:

$$\text{Program} = \text{Intent} + \text{Model priors} + \text{Repository} + \text{Dependencies} + \text{Examples} + \text{Tools} + \text{Tests}$$

Это **моя синтетическая модель**, а не принятая формула, но она хорошо соответствует тому, как современные coding agents реально работают: Anthropic, например, описывает context engineering через выбор и сжатие relevant history, загрузку файлов, использование tools и сохранение архитектурных решений между этапами работы. ¹⁶

Именно поэтому рождается прекрасная фраза:

Короткий prompt — это не маленькая программа. Это маленький указатель на огромную программу, распределённую между моделью, репозиторием и культурой разработки.

И ещё сильнее:

Мы не увеличили смысл букв. Мы увеличили объём контекста, который способен восстановить получатель этих букв.

Это похоже на compression, но я бы специально называл это **semantic compression как метафору**, а не пытался привязать формально к Shannon entropy.

«Как вот там» становится полноценным programming construct

Это очень важное следствие.

В классическом языке нельзя написать:

```
BuildThisFormLike(TheNiceOneWeAlreadyHave);
```

если кто-то заранее не реализовал такую функцию.

Agent может получить:

«Сделай как `CustomerDetails`, только для `supplier`».

И для него имя существующего экрана фактически становится **макросом гигантского размера**.

Одна ссылка на component или file может кодировать:

- архитектуру;
- UX;
- naming;
- layout;
- validation;
- error handling;
- styling;
- conventions.

Современные рекомендации по context engineering именно поэтому делают акцент на предоставлении агенту небольших identifiers и возможности самостоятельно подтягивать нужный контекст, а не на помещении всего возможного знания в один гигантский prompt. ¹⁷

Отсюда ещё одна отличная сценическая формулировка:

Reference is the highest-bandwidth prompt.

И на русском:

Лучший способ долго объяснять — не объяснять, а показать, на что ты ссылаешься.

Это напрямую отвечает на вашу дилемму «либо коротко, либо точно».

Третий вариант:

коротко + shared context.

Модель уже не просто переводит — она заполняет недосказанное

Компилятор не должен догадываться:

«Наверное, Дмитрий хотел здесь authorization».

Agent постоянно делает именно такие inference.

Это огромное изменение.

Поэтому natural language можно считать программой только вместе с **набором implicit defaults**.

И здесь полезен terminology research: в свежей литературе уже встречаются термины **prompt programming, natural language programming** и **promptware**. В исследовании *Prompts Are Programs Too!* разработчики описывали prompt development как отдельный вид программирования; интересно, что даже участники, много раз создававшие и корректировавшие prompts, испытывали трудности с построением устойчивой mental model поведения foundation model. ¹⁸

А к 2026 году уже появляются работы, пытающиеся снова добавить к prompt programs привычные PL concepts — compilation, structured representations и typing. Например, Imprompt прямо рассматривает prompt program как task description, отделённое от низкоуровневых execution details. ¹⁹

Это фантастически красиво ложится в историческую картину:

Сначала мы избавились от формального языка.
Потом обнаружили, что жить совсем без структуры неудобно.
И начали изобретать типы для английского.

То есть будущее, скорее всего, не выглядит как:

```
code → English
```

а скорее:

```
code
  ↓
intent
+ context
+ references
+ constraints
+ executable specifications
+ tests
```

И это уже **новый programming stack**.

Парадокс ограничений: почему свобода LLM одновременно мешает

Здесь ваша интуиция тоже очень хорошая, но я бы немного изменил Goldratt angle.

Theory of Constraints Голдратта — не теория о том, что ограничения повышают человеческую креативность. ТОС прежде всего предлагает найти наиболее важное системное ограничение — bottleneck, препятствующее достижению цели, — и работать с ним. Поэтому Голдратта лучше использовать не как психологическое доказательство, а как метафору того, **куда переехал bottleneck разработки.** 20

Для собственно creative constraints есть более интересный и релевантный software-engineering материал.

В двух randomized experiments исследователи проверяли, как влияет формулировка desiderata для software design. Когда те же желания предъявлялись как более формальные requirements, решения становились **практичнее, но менее оригинальными**; сама framing задачи влияла на design outcome. 21

Это почти готовый эксперимент про prompting, проведённый ещё до нынешней agent era.

И я бы сформулировал вывод очень аккуратно:

Ограничения не просто улучшают результат. Они меняют распределение возможных результатов.

Сделать это можно визуально.

Огромный конус вариантов



Потом:

«Сделай форму клиента
используя существующий CustomerForm,
FluentValidation,
наши design tokens,
без новых dependencies,
WCAG AA,
все acceptance tests должны пройти»

И конус схлопывается.

Математическая картинка для programmer audience:

$$S_{result} = S_{model} \cap C_{architecture} \cap C_{design} \cap C_{tests} \cap C_{performance} \cap C_{business}$$

И реплика:

Хороший prompt не обязательно добавляет информацию. Иногда он просто отнимает неправильные варианты.

Ещё сильнее:

Constraint — это способ сообщить модели, что она не имеет права иметь в виду.

Это удачная формулировка именно для вашего тезиса про meaning.

Поэтому «самый короткий prompt» — неправильная цель

Если измерять качество programming interface количеством символов, победит:

Do it.

Но его семантическая плотность практически бессмысленна без контекста.

Правильнее оптимизировать не:

$$\frac{output}{tokens}$$

а нечто вроде:

$$\frac{acceptable\ output}{explicit\ clarification\ effort}$$

То есть выигрывает не минимальный prompt.

Выигрывает **минимальная спецификация, которая достаточно хорошо ограничивает пространство неправильных реализаций.**

И здесь можно ввести очень полезное различие.

Низкоплотное уточнение	Высокоплотное уточнение
«Кнопка должна иметь radius 6 px, padding 8 на 16...»	«Используй наш PrimaryButton»
длинное описание layout	«Как на CustomerDetails»

Низкоплотное уточнение	Высокоплотное уточнение
описание правил валидации словами	существующие validators
объяснение architecture	ссылка на соседний feature
«не сломай старое»	regression tests
длинное описание style	design system / tokens

Вот, на мой взгляд, **практический ответ на центральную проблему доклада**:

Будущее prompt engineering — не в том, чтобы научиться писать очень хорошие длинные тексты. Оно в том, чтобы построить хороший shared context, на который достаточно коротко указывать.

Это подтверждается и практикой agent systems: Anthropic прямо называет relevant-context selection и context compression центральной инженерной проблемой для агентов, а не просто «написанием лучшего prompt». ¹⁷

Самая неожиданная вещь: ограничения снова возвращают формальность

Очень красивый исторический цикл:

```

Machine code
  ↓
Formal symbolic languages
  ↓
Higher-level formal languages
  ↓
Declarative formal languages
  ↓
Natural language
  ↓
???
```

Первое ощущение:

конец формальных языков.

Но реальные agent workflows добавляют назад:

- schemas;
- types;
- test suites;
- linters;
- policies;

- acceptance criteria;
- tool permissions;
- structured outputs.

То есть я бы предсказание для доклада сформулировал так:

Код не исчезнет из-за natural language. Код станет системой ограничений для natural language.

Ещё более сценично:

У программиста будущего будет два языка: язык, на котором он говорит, чего хочет, и язык, на котором он доказывает, что получил именно это.

Например:

Intent:

«Добавь возможность отменить заказ».

Executable boundary:

```
[Fact]
public async Task Paid_order_cannot_be_cancelled_after_shipping()
```

И фраза:

Тест — это prompt, с которым модель не может спорить.

Как буквальное определение это, конечно, метафора. Но как conference line — прекрасная.

Сдвиг бутылочного горлышка: вот где на самом деле «программирование уже не будет прежним»

Самая слабая версия AI-доклада:

«Теперь AI пишет код быстрее».

Самая сильная:

Кодогенерация перестаёт быть главным дефицитным ресурсом. Поэтому дефицитными становятся specification, context и judgment.

Это можно обосновать сразу несколькими неожиданными исследованиями.

Генерация кода и производительность — не одно и то же

Летом 2025 года METR провёл randomized controlled trial с 16 опытными open-source developers, выполнявшими 246 реальных задач в repositories, которые они хорошо знали. С доступными тогда AI tools задачи в среднем заняли **на 19% больше времени**, чем без AI. ²²

Самое смешное — и сценически великолепное — разработчики ожидали ускорения примерно на 24%, а после выполнения задач всё ещё считали, что AI ускорил их примерно на 20%, хотя измерение показало обратное. ²³

На слайде:

Как разработчикам казалось:	+20% быстрее
Что показал эксперимент:	19% медленнее

И пауза.

«Наконец-то у нас появился benchmark developer confidence».

Но здесь обязательно нужна честная актуализация.

В феврале 2026 года METR сообщил, что теперь считает ускорение от более новых AI tools вероятным, однако новый эксперимент оказался серьёзно испорчен selection effects: часть разработчиков уже отказывалась участвовать, если им приходилось выполнять задачи **без AI**. Поэтому исследователи не считают новые данные надёжной оценкой величины ускорения. ²⁴

Это даже интереснее старого результата.

Фраза:

AI меняется настолько быстро, что control group начала отказываться жить в прошлом.

Именно так я бы использовал исследование: не как «AI бесполезен», а как доказательство того, что **скорость генерации и эффективность всей инженерной системы — разные величины**.

Generation дешёвая, evaluation остаётся дорогой

Это напрямую соединяется с классическим HCI.

Norman и коллеги называют **gulf of execution** расстоянием между целью пользователя и действиями, которыми её можно выразить системе, а **gulf of evaluation** — трудностью понять, что система сделала и соответствует ли результат цели. ²⁵

И вот здесь возникает очень сильная **моя интерпретация** применительно к LLM:

Coding agents радикально сокращают **gulf of execution**.
Но могут увеличить нагрузку на **gulf of evaluation**.

Раньше:

```
Intent
  ↓↓↓↓
много ручной реализации
  ↓
Result
```

Теперь:

```
Intent
  ↓
Agent
  ↓↓↓↓↓↓↓↓
огромный Result
  ↓↓↓↓
Verification
```

Очень сценическая формулировка:

Мы решили проблему скорости написания кода — и обнаружили проблему скорости чтения кода.

И:

Если машина способна писать десять тысяч строк быстрее, чем человек способен их понять, bottleneck не исчез. Он просто переехал вправо.

Это не установленная количественная закономерность, а логический вывод из agent workflow и результатов productivity studies. ²⁶

Цена делегирования — потеря части внутренней модели

Ещё один очень свежий и мощный факт.

В январе 2026 года Anthropic опубликовал randomized experiment с 52 разработчиками, изучавшими незнакомую им Python library. Участники с AI assistance показали на последующем knowledge quiz результат примерно **на 17% ниже**, чем группа, писавшая самостоятельно; наблюдавшееся ускорение выполнения задания при этом не достигло statistical significance. Исследователи также обнаружили,

что эффект сильно зависел от поведения: пользователи, которые задавали conceptual questions и использовали AI для понимания, сохраняли знания лучше. 27

Для слайда:

Abstraction hides complexity. Sometimes it also hides learning.

И продолжение:

«Мы всегда платили за abstraction потерей visibility.
Но раньше за abstraction стоял deterministic compiler.
Теперь между нами и реализацией находится ещё и система, способная принять сотни проектных решений без нашего непосредственного участия».

Это прекрасно связывается с **Law of Leaky Abstractions** Джоэла Спольски: его формулировка 2002 года состоит в том, что нетривиальные abstractions в той или иной степени leak, поэтому знание нижнего уровня время от времени всё равно требуется. 28

LLM не отменяет закон.

Он делает leak потенциально значительно дальше от уровня, на котором человек сформулировал намерение.

Prompt:
"Добавь login через Google"

↓ leak

UI
OAuth
cookies
redirects
state
PKCE
DB
identity mapping
security
session lifetime
CSRF
error handling
...

Фраза:

Чем выше abstraction, тем дальше приходится падать, когда она протекает.

И здесь возвращается Brooks

Вот почему *No Silver Bullet* стоит сделать эмоциональным центром доклада.

Брукс считал, что тяжёлая часть software engineering — specification, design и testing сложной conceptual structure, а не механическое выражение этой структуры в programming language. Он даже подчёркивал, что более высокие abstractions позволяют убрать syntactic material, который сам по себе не добавляет information content, но не устраняют сложность design. ²⁹

В 1986 году это выглядело почти философски.

В 2026 году можно буквально провести эксперимент:

Build me a CRM.

Модель способна произвести много тысяч строк.

Проблема мгновенно становится очевидной:

«Какой CRM?»

Вот здесь вы и возвращаетесь к началу:

чем больше смысла способна разворачивать одна фраза, тем дороже становится ошибка в этой фразе.

Это очень сильный новый тезис.

Не просто:

meaning per character растёт.

А:

responsibility per character тоже растёт.

Можно даже закончить этим.

Сценарий доклада на двадцать минут

Я бы делал не больше **двенадцати основных слайдов**. Несколько должны существовать на экране по 20–30 секунд; исторический материал нельзя превращать в лекцию по истории языков.

Главное dramaturgical правило: **первые семь минут зритель думает, что это доклад об abstraction; на восьмой минуте он понимает, что на самом деле это доклад о смене роли программиста.**

Время	Слайд	Что на экране	Что вы говорите
0:00–1:10	Одна задача. Пять эпох.	Проводка ENIAC → machine/assembly → <code>for</code> → SQL → «Добавь форму клиента»	«Всю историю мы делали одну и ту же вещь: пытались сказать машине меньше, чтобы она сделала больше».
1:10–2:10	Я назвал это энтропией	Большое <code>ENTROPY</code> → зачёркнуто → <code>SEMANTIC DENSITY</code>	«Потом выяснилось, что Шеннон специально попросил не путать информацию со смыслом». ¹
2:10–3:30	Сначала программа была машиной	ENIAC / cables / punch card	«Первая abstraction layer выглядела примерно как электрический шкаф». Исторически ENIAC программировали конфигурацией switches и wiring. ³⁰
3:30–5:00	Мы начали переносить работу в interpreter	machine → assembly → FORTRAN	Расскажите про FORTRAN translator и огромную инженерную инвестицию в compiler, позволившую последующим программистам писать меньше machine detail. ³¹
5:00–6:20	Каждый слой строит новую машину	Dijkstra staircase	«На CPU нет <code>Customer</code> . Мы строили машины, для которых Customer существует». ⁹
6:20–7:30	SQL: скажи what, отдай how	SQL query + скрывающийся execution machinery	Relational model усилил data independence и отделение запроса от physical representation. ¹⁰
7:30–9:00	1986: Brooks spoils the ending	Только портрет/название статьи и одна строка	Показать его фразу про «deciding what one wants to say». Пауза. «Сорок лет назад в разделе про AI человек уже сформулировал проблему 2026 года». ⁴

Время	Слайд	Что на экране	Что вы говорите
9:00–10:40	Мы не сжали программу. Мы перенесли её в интерпретатор	Prompt + Model + Repo + Tools + Tests	Главная новая модель: одна короткая фраза работает только благодаря гигантскому скрытому context. Agent systems явно собирают и сжимают такой context. ¹⁶
10:40–12:40	Свобода — это баг	Большой конус решений → узкий конус после constraints	Один prompt порождает огромное reasonable solution space. Constraints/reference/tests его сужают. Requirements-framing research показывает реальный trade-off между originality и practicality. ²¹
12:40–14:20	Reference > Description	300 слов style instructions vs Follow CustomerDetails	«Самая высокая semantic density получается не из гениальных prompt-фраз, а из shared context». Context engineering современных agents построен вокруг selective retrieval. ¹⁷
14:20–16:50	Bottleneck moved	METR ^{ожидали} быстрее → получили медленнее ; затем 2026 update	Генерация ≠ productivity. 2025 RCT: +19% time; к 2026 newer tools, вероятно, дают acceleration, но измерение уже мешает widespread adoption. ³²
16:50–18:10	Теперь дорого понимать	Gulf of execution ↓, Gulf of evaluation ↑	«Мы сократили путь от намерения к artifact. Теперь нам надо успевать понять artifact». Используйте 17%-lower-mastery experiment как provocative evidence, не как универсальный закон. ³³
18:10–20:00	Новая единица программирования — намерение	Instruction → Statement → Function → Query → Intent	«Программирование не исчезает. Оно меняет свою минимальную единицу. А вместе с ней меняется ответственность программиста».

Как сделать первый слайд действительно сильным

Не показывать пять определений.

Покажите **одну и ту же feature в пяти представлениях**.

Например:

Уровень машины

```
load...  
compare...  
branch...  
increment...
```

Не пытайтесь давать настоящий огромный assembly listing — аудитория начнёт его читать и проверять.

Уровень imperative

```
foreach (var customer in customers)  
{  
    if (customer.Orders.Count == 5)  
        result.Add(customer);  
}
```

Уровень declarative

```
...  
HAVING COUNT(*) = 5
```

Уровень agent

На странице клиентов добавь фильтр:
«ровно пять заказов».
Сделай так же, как остальные фильтры.

И затем:

«Смотрите не на количество строк.
Смотрите на количество решений, которые исчезли из текста».

Это мгновенно объясняет вашу идею.

Как сделать главный reveal

На слайде с agent prompt сначала показать только:

Добавь форму редактирования клиента.

Спросить:

«Сколько информации здесь?»

Пусть аудитория мысленно ответит: 4–5 слов.

Следующий click:

Добавь форму редактирования клиента.

- + Claude/GPT/... weights
- + весь публичный код, увиденный при обучении
- + framework conventions
- + repository
- + git history
- + design system
- + nearby screens
- + tests
- + tools

И:

«Вот теперь мы видим настоящую длину программы.»

В строгом смысле model weights, training experience и repository не являются частью source program, поэтому на сцене лучше говорить не «это программа» как formal claim, а:

«Вот инфраструктура смысла, благодаря которой эти пять слов вообще что-то означают».

Это интеллектуально аккуратнее.

Демонстрация, которая может стать самым запоминающимся моментом

Я бы не делал live coding. Риск непропорционально высок.

Сделайте заранее **три screenshots** ответа агента на один и тот же prompt:

«Сделай форму редактирования клиента».

Например:

- один conventional Bootstrap-like;
- один minimalist;
- один dashboard-like.

Три картинки рядом.

Подпись:

Same program?

Затем второй слайд:

«Сделай форму клиента так же, как `OrderDetails`. Используй существующие form components. Никаких новых dependencies. Existing tests must pass.»

И три результата должны стать значительно ближе.

Главная реплика:

«Я не столько рассказал машине, что сделать. Я лишил её права сделать всё остальное».

Это прекрасное визуальное доказательство вашей части про constraints.

Банк вау-материала, из которого стоит выбирать

Нижe я бы оценивал материал не по академической важности, а именно по способности **остановить аудиторию на несколько секунд**.

Материал	Вау	Использовать?	Почему
Brooks, 1986, раздел AI: hard part — решить, что сказать	★★★★★	Обязательно	Почти пугающе точно описывает эпоху coding agents. 4
Shannon специально отделяет information от semantics	★★★★★	Обязательно	Отличный self-deprecating opening и интеллектуальная честность. 1
«Мы не сжали программу; мы перенесли её в интерпретатор»	★★★★★	Обязательно	Это наиболее компактная версия всей вашей идеи. Это синтез исследования, не цитата.
ENIAC программируется проводами и switches	★★★★★	Да	Визуально мгновенно создаёт дистанцию между эпохами. 5

Материал	Вай	Использовать?	Почему
IBM punch card ≈ одна строка кода	★★★★☆	Да, ради шутки	«Git diff имел вес». ⁶
FORTTRAN compiler стоил огромной разовой инженерной работы	★★★★☆	Да	Демонстрирует, что краткость пользователя оплачивается сложностью interpreter. ³⁴
Dijkstra: software layer создаёт следующую abstract machine	★★★★★	Да	Лучшее академическое основание вашей лестницы abstractions. ³⁵
Codd: отделение logical data от machine representation	★★★★☆☆	Коротко	SQL все знают; важна не история SQL, а принцип delegation. ¹⁰
Simonyi, Intentional Programming, 1990-e	★★★★☆	Да, если аудитория сильная	Показывает, что intent-first programming старше LLM. ¹²
Лебедев, «Единица смысла», 2008	★★★★☆	Да для русскоязычной аудитории	Неожиданная культурная отсылка ровно к вашей формулировке. ¹³
Function points измеряют functionality, а не LOC	★★★★☆☆	Backup slide	Поддерживает идею давнего поиска метрик выше уровня текста. ³⁶
Prompts are Programs Too — разработчики плохо формируют устойчивую mental model LLM	★★★★☆	Да	Natural-language programming уже изучается как отдельный вид development. ¹⁸
Promptware: NL ambiguous/context-dependent, runtime probabilistic	★★★★☆	Да	Научное описание главного различия code vs prompt. ¹⁵
Requirements framing: больше формальности → практичнее, но менее оригинально	★★★★★	Очень желательно	Неожиданная эмпирика для вашего раздела constraints. ²¹
METR: опытные developers в 2025 были 19% медленнее с AI	★★★★★	Да, но вместе с update	Разрушает простой narrative «больше generated code = productivity». ²²
Они сами всё равно считали, что ускорились	★★★★★	Да	Идеальный conference laugh + reflection. ²³

Материал	Bay	Использовать?	Почему
В 2026 METR уже считает acceleration более вероятным, но experiment ломает adoption	★★★★★	Обязательно после предыдущего	Показывает честность и абсурдную скорость изменения индустрии. ²⁴
AI-learning experiment: -17% mastery	★★★★★	Да ближе к финалу	Поднимает разговор от productivity к изменению expertise. ²⁷
Law of Leaky Abstractions	★★★★☆	Да одной фразой	Идеальная тёмная сторона роста abstraction. ²⁸
Gulf of execution / gulf of evaluation	★★★★☆	Да	Даёт название новому bottleneck: генерировать проще, проверять всё ещё сложно. ²⁵
Anthropic: доля GitHub projects с coding-agent activity более чем удвоилась с конца 2025	★★★☆☆	Только если нужен current-state слайд	Подтверждает скорость adoption, но это vendor telemetry. ³⁷
Claude Code users в том же исследовании — около 20 часов использования в неделю	★★★☆☆	Необязательно	Хорошо показывает, что agent coding становится рабочей средой, но не добавляет главному тезису столько же, сколько Brooks/METR. ³⁷

Я бы **не пытался вставить всё**. Для двадцати минут Tier A — это Shannon, ENIAC, FORTRAN, Dijkstra, Brooks, agent/context, constraints experiment, METR, mastery study, final thesis. Остальное — backup slides и материал для вопросов.

Формулировки, которые могут сделать доклад похожим на TED, а не на обзор технологий

Ваш доклад выиграет, если повторять **одну и ту же мысль разными способами**, постепенно меняя её смысл.

В начале:

История программирования — это история того, как мы учились говорить компьютеру меньше.

Через пять минут:

Каждый новый уровень abstraction — это класс решений, которые программист больше не обязан принимать.

После FORTRAN:

Краткость программы всегда кем-то оплачена. Обычно — сложностью интерпретатора.

После Dijkstra:

Мы не столько изобретали новые языки, сколько строили новые воображаемые компьютеры.

Перед LLM:

На CPU нет клиентов, кнопок и заказов. Мы потратили восемьдесят лет, чтобы убедить компьютер, что они существуют.

На LLM reveal:

Natural language оказался не самым выразительным языком. Он оказался языком с самым большим runtime.

Это метафора, но очень хорошая.

После формулы context:

Мы не сжали десять тысяч строк кода в двенадцать слов. Мы спрятали недостающие девять тысяч девятьсот восемьдесят восемь строк в shared knowledge.

Число здесь лучше использовать явно как художественную гиперболу, а не статистику.

Про common sense:

Common sense — это новая standard library.

Опять же метафора, но прекрасная.

Про ссылки:

Когда у нас есть общий контекст, имя становится compression algorithm.

Про design system:

`Use our design system` — это одна строка, за которой стоят тысячи уже принятых решений.

Про constraints:

Хорошая спецификация сообщает не только, чего мы хотим. Она уничтожает тысячи вещей, которых мы не хотим.

Или короче:

Constraints compress the search space.

Про tests:

В новой разработке тесты нужны не только для проверки кода. Они объясняют агенту границы значения слова «готово».

Это, на мой взгляд, одна из самых практически полезных мыслей доклада.

Про Brooks:

Сорок лет мы автоматизировали “saying it”. И вот наконец упёрлись в “knowing what to say”.

Про agent productivity:

Написать код становится дешёво. Доказать, что это правильный код, — нет.

Про новое bottleneck:

Когда production становится бесконечно дешёвым, review становится производством.

Про роль programmer:

Раньше программист переводил намерение человека на язык машины. Теперь машина умеет переводить. Программист остаётся ответственным за намерение.

Про future skills:

Архитектор будущего проектирует не только software. Он проектирует пространство допустимых решений для агента.

Про leaky abstractions:

Чем выше abstraction, тем меньше вы видите в хороший день — и тем дальше приходится падать в плохой. 28

Про semantic density:

Каждая следующая эпоха давала программисту больше власти на символ. LLM впервые дала ему настолько много власти, что главным риском стало неточно подобрать символы.

И мой фаворит для предпоследнего слайда:

Когда одна фраза может породить десять тысяч строк кода, `typing` перестаёт быть `bottleneck`. `Meaning` становится `bottleneck`.

Возможный финал

Я бы убрал с последнего слайда почти всё.

Только:

1946	instruction
1957	statement
1970	query
2026	intent

И речь примерно такая:

«Мы привыкли думать, что программирование — это умение объяснять компьютеру, **как** что-то сделать.

Но большую часть истории нашей профессии мы на самом деле занимались противоположным.

Мы строили всё более мощные слои, чтобы больше **не объяснять как**.

Сначала мы перестали выбирать числовые коды инструкций.

Потом перестали выбирать регистры.

Потом перестали управлять памятью.

Потом перестали выбирать физический способ доступа к данным.

Теперь мы начинаем переставать писать саму реализацию.

И можно решить, что на этом программирование заканчивается.

Мне кажется — наоборот.

Просто впервые за восемьдесят лет исчезает шум, за которым пряталась его самая сложная часть.

Брукс написал это ещё в 1986 году: трудность — не сказать компьютеру, а решить, что именно мы хотим сказать. ⁴

Чем меньше кода нам придётся писать, тем больше ответственности будет приходиться на каждое оставшееся слово.

Поэтому программирование действительно никогда уже не будет прежним.

Его новой единицей становится не инструкция. Не строка. Даже не функция. Его новой единицей становится намерение».

Пауза.

И совсем последняя строка на экране:

Less code. More meaning. More responsibility.

Это сохраняет вашу исходную идею «больше смысла в одной буковке», но выводит её из уровня «AI теперь может написать форму по тексту» к значительно более сильному утверждению:

Вся история software engineering была процессом повышения semantic leverage человека. Coding agents не ломают эту историю — они доводят её до момента, когда ограничивающим ресурсом становится уже не способность выразить решение, а способность достаточно точно сформулировать, проверить и отстоять собственное намерение. Это хорошо согласуется и с классическим разделением essential/accidental complexity у Брукса, и с современной практикой context-driven agents, и с пока неоднозначными эмпирическими результатами исследований AI-assisted development. ³⁸

📖navlist📖Недавние материалы о сдвиге в разработке с AI📖turn16news41,turn16news42,turn16news43📖

¹ <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>
<https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>

² <https://www.anthropic.com/engineering/building-agents-with-the-claude-agent-sdk>
<https://www.anthropic.com/engineering/building-agents-with-the-claude-agent-sdk>

- 3 4 29 38 <https://www.cs.unc.edu/techreports/86-020.pdf>
<https://www.cs.unc.edu/techreports/86-020.pdf>
- 5 30 <https://www.computerhistory.org/timeline/computers/>
<https://www.computerhistory.org/timeline/computers/>
- 6 8 <https://www.ibm.com/history/punched-card>
<https://www.ibm.com/history/punched-card>
- 7 <https://dl.acm.org/doi/10.1145/1455567.1455599>
<https://dl.acm.org/doi/10.1145/1455567.1455599>
- 9 <https://www.cs.utexas.edu/~EWD/transcriptions/EWD01xx/EWD196.html>
<https://www.cs.utexas.edu/~EWD/transcriptions/EWD01xx/EWD196.html>
- 10 https://db.dobo.sk/wp-content/uploads/2015/11/Codd_1970_A_relational_model.pdf
https://db.dobo.sk/wp-content/uploads/2015/11/Codd_1970_A_relational_model.pdf
- 11 <https://lapets.io/course-programming-languages/>
<https://lapets.io/course-programming-languages/>
- 12 <https://www.usenix.org/conference/dsl-97/invited-talk-intentional-programming-ecology-abstractions>
<https://www.usenix.org/conference/dsl-97/invited-talk-intentional-programming-ecology-abstractions>
- 13 <https://www.artlebedev.ru/kovodstvo/sections/148/>
<https://www.artlebedev.ru/kovodstvo/sections/148/>
- 14 36 <https://ifpug.org/about-us>
<https://ifpug.org/about-us>
- 15 <https://arxiv.org/abs/2503.02400>
<https://arxiv.org/abs/2503.02400>
- 16 17 <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- 18 <https://arxiv.org/abs/2409.12447>
<https://arxiv.org/abs/2409.12447>
- 19 <https://arxiv.org/abs/2607.22683>
<https://arxiv.org/abs/2607.22683>
- 20 <https://www.leanproduction.com/theory-of-constraints/>
<https://www.leanproduction.com/theory-of-constraints/>
- 21 https://www.researchgate.net/publication/328486398_Requirements_Framing_Affects_Design_Creativity
https://www.researchgate.net/publication/328486398_Requirements_Framing_Affects_Design_Creativity
- 22 26 32 <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>
<https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>
- 23 <https://www.reuters.com/business/ai-slows-down-some-experienced-software-developers-study-finds-2025-07-10/>
<https://www.reuters.com/business/ai-slows-down-some-experienced-software-developers-study-finds-2025-07-10/>
- 24 <https://metr.org/blog/2026-02-24-uplift-update/>
<https://metr.org/blog/2026-02-24-uplift-update/>

25 33 <https://www.nngroup.com/articles/two-ux-gulfs-evaluation-execution/>
<https://www.nngroup.com/articles/two-ux-gulfs-evaluation-execution/>

27 <https://www.anthropic.com/research/AI-assistance-coding-skills>
<https://www.anthropic.com/research/AI-assistance-coding-skills>

28 <https://www.joelonsoftware.com/>
<https://www.joelonsoftware.com/>

31 34 [https://www.cs.umass.edu/~emery/classes/cmpsci691st/readings/PL/](https://www.cs.umass.edu/~emery/classes/cmpsci691st/readings/PL/FORTRAN-102663113.05.01.acc.pdf)
[FORTRAN-102663113.05.01.acc.pdf](https://www.cs.umass.edu/~emery/classes/cmpsci691st/readings/PL/FORTRAN-102663113.05.01.acc.pdf)
<https://www.cs.umass.edu/~emery/classes/cmpsci691st/readings/PL/FORTRAN-102663113.05.01.acc.pdf>

35 <https://www.cs.utexas.edu/~EWD/transcriptions/EWD02xx/EWD236.html>
<https://www.cs.utexas.edu/~EWD/transcriptions/EWD02xx/EWD236.html>

37 <https://www.anthropic.com/research/claude-code-expertise>
<https://www.anthropic.com/research/claude-code-expertise>